

Week1_예습과제_진예원

1장 머신 러닝과 딥러닝

1.1 인공지능, 머신 러닝과 딥러닝

1. 인공지능

- 정의: 인간의 지능을 모방하여 사람이 하는 일을 컴퓨터(기계)가 할 수 있도록 하는 기술
- 구현 방법: 머신 러닝, 딥러닝
- 관계: 인공지능(전문가 시스템) $\supset$ 머신 러닝 $\supset$ 딥러닝

2. 머신 러닝과 딥러닝

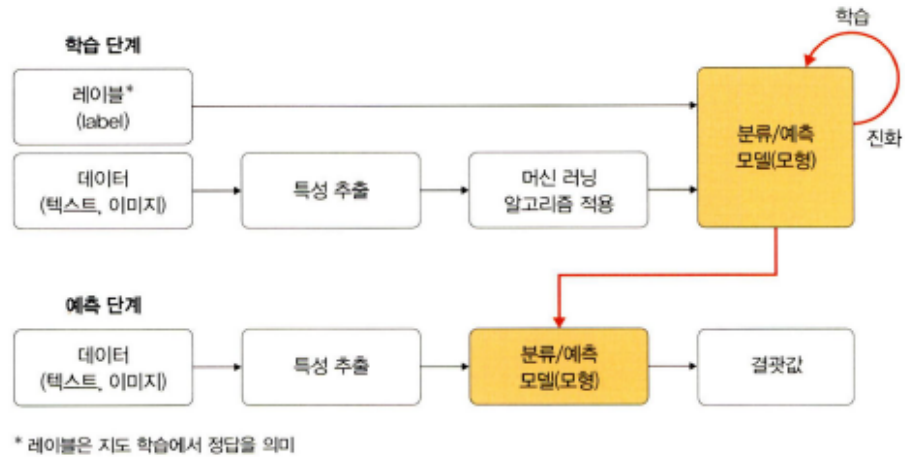
- 공통점: 학습 모델을 제공하여 데이터 분류
- 차이점: 머신 러닝 - 주어진 데이터를 인간이 먼저 처리(전처리) 범용적 목적, 데이터 특징 스스로 추출 X. 각 데이터(or 이미지) 특성을 컴퓨터(기계)에 인식시키고 학습시켜 문제 해결
딥러닝 - 전처리 생략. 대량 데이터를 신경망에 적용하면 컴퓨터가 스스로 분석 후 답을 찾음

1.2 머신 러닝이란

1.2.1 머신 러닝 학습 과정

1. 머신 러닝 = 학습 단계 + 예측 단계

- 학습 단계: 훈련 데이터를 머신 러닝 알고리즘에 적용하여 학습 $\rightarrow$ 모형 생성
- 예측 단계: 생성 모형에 새로운 데이터 적용하여 결과 예측
- * 특성 추출: 사람이 인지하는 데이터 $\Rightarrow$ 컴퓨터가 인지할 수 있는 데이터 변환시, 데이터별 특징 찾아내고, 데이터를 벡터로 변환하는 작업



2. 구성 요소

1) 데이터: 학습 모델 만드는 데 사용하는 것. 훈련 데이터 나쁘다면 실제 현상 특성 제대로 반영 X → 실제 데이터 특징 잘 반영되고 편향되지 않는 훈련 데이터 확보 중요.

수집 이후, 데이터 = 훈련 데이터셋 + 테스트 데이터셋으로 분리해서 사용. or 훈련 데이터셋 = re 훈련 데이터셋 + 검증 데이터셋으로 분리해서 사용. 보통 데이터 80% = 훈련용, 20% = 테스트용

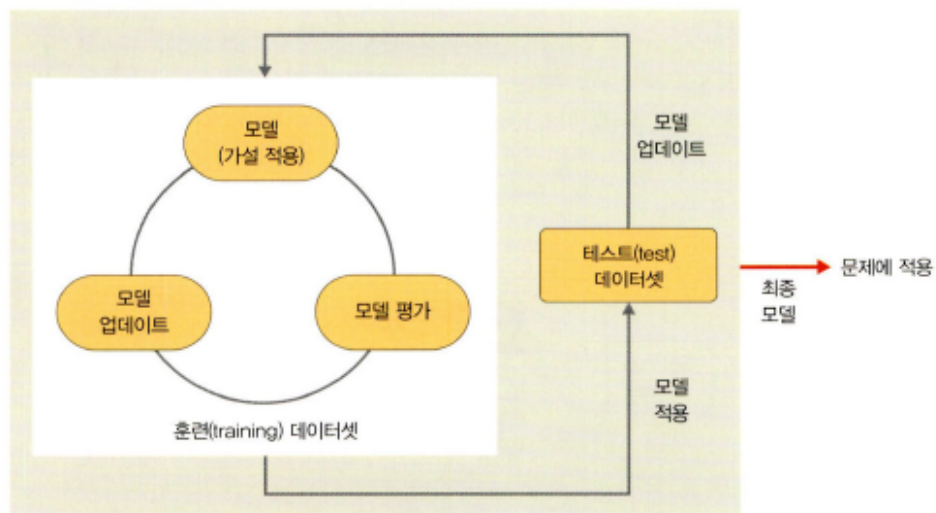
* 검증 데이터셋 목적: 모델 성능 평가

2) 모델: 학습 단계에서 얻은 최종 결과물 = 가설

- 학습 절차: 모델(또는 가설) 선택 → 모델 학습 및 평가 → 평가를 바탕으로 모델 업데이트

세 단계 반복하며 주어진 문제 가장 잘 풀 수 있는 모델 찾음

▼ 그림 1-5 머신 러닝의 문제 풀이 과정



1.2.2 머신 러닝 학습 알고리즘

1. 지도 학습: 정답이 무엇인지 컴퓨터에 알려 주고 학습시키는 방법

2. 비지도 학습: 정답 알려주지 않고 특징을 클러스터링(범주화)하여 예측하는 학습 방법
 * 지도는 주어진 데이터에 대해 A or B로 명확한 분류 가능 / 비지도는 유사도 기반(데이터 간 거리 측정)으로 특징 유사한 데이터끼리 클러스터링으로 묶어서 분류



3. 강화 학습: 분류할 수 있는 데이터 X 데이터 있다고 해도 정답 X. 자신의 행동에 대한 보상 받으며 학습 진행. 행동에 따라 보상을 얻으며 보상이 커지는 행동은 자주 하도록, 줄어드는 행동은 덜 하도록 하여 학습 진행

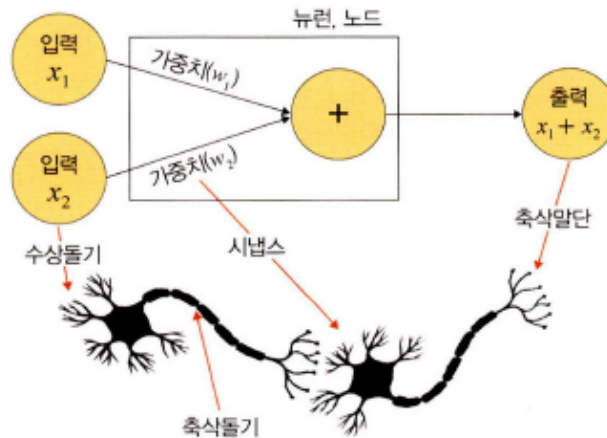
1.3 딥러닝이란

1. 정의 및 특징

- 정의: 인간의 신경망 원리를 모방한 심층 신경망 이론으로 고안된 머신 러닝 방법 중 하나
- 머신 러닝과의 차이점: 인간의 뇌를 기초로 하여 설계했다는 점. 인간의 뇌가 엄청난 수의 뉴런과 시냅스로 구성되어 있는 것에 착안하여 적용. 각각의 뉴런은 복잡하게 연결된 많은 뉴런을 병렬 연산하여 음성, 영상 인식 등 처리 가능하게 함.

2. 뉴런 구성 요소

- 1) 수상돌기: 주변이나 다른 뉴런에서 자극 받아들이고, 이 자극들을 전기적 신호 형태로 세포체와 축삭돌기로 보냄
- 2) 시냅스: 신경 세포들이 이루는 연결 부위로, 한 뉴런의 축삭돌기와 다음 뉴런의 수상돌기가 만나는 부분
- 3) 축삭돌기: 다른 뉴런(수상돌기)에 신호 전달 기능하는 뉴런의 한 부분. 뉴런에서 뻗어 있는 돌기 중 가장 길고, 한개만 존재
- 4) 축삭말단: 전달된 전기 신호를 받아 신경 전달 물질을 시냅스 틈새로 방출함.



1.3.1 딥러닝 학습 과정

1. 데이터 준비

- 1) 파이토치(<https://tutorials.pytorch.kr/>)나 케라스(<https://keras.io/>)에서 제공하는 데이터셋 사용 - 전처리 되어있음, 여러 예제 코드 쉽게 구할 수 O
- 2) 캐글(Kaggle)에 공개된 데이터 사용

2. 모델(모형) 정의

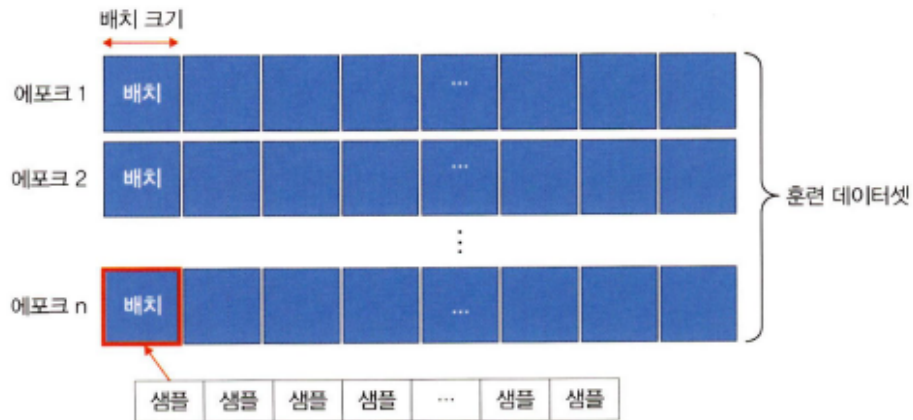
- 신경망 생성
- 일반적으로 은닉층 개수 $\uparrow \rightarrow$ 성능 $\uparrow$ But 과적합 발생 $\uparrow$ (은닉층 개수에 따른 성능 $\leftrightarrow$ 과적합)

3. 모델(모형) 컴파일

- 활성화 함수, 손실 함수, 옵티마이저 선택 $\leftarrow$ (훈련) 데이터 형태에 따른 옵션 존재
- 1) 연속형 - 평균 제곱 오차(MSE)
- 2) 이진 분류 - 크로스 엔트로피

4. 모델(모형) 훈련

- 한 번에 처리할 데이터양 지정
- if 데이터양 많음 $\rightarrow$ 학습 속도 $\downarrow$, 메모리 부족
- $\Rightarrow$ 적당한 데이터양 선택이 중요
- $\Rightarrow$ 전체 훈련 데이터셋에서 일정한 묶음으로 나누어 처리할 수 있는 배치와 훈련 횟수인 에포크 선택이 중요
- 훈련 과정에서 값의 변화를 시각적으로 표현하여 눈으로 확인하면서 파라미터와 하이퍼파라미터에 대한 최적의 값을 찾을 수 있어야 함
- * 훈련 = 학습
- ex) 훈련 데이터셋 1000개에 대한 배치 크기가 20 - 샘플 단위 20개마다 모델 가중치를 한 번씩 업데이트 시킨다. 에포크 10, 배치 크기 20 $\rightarrow$ 가중치 50번 업데이트하는 것을 총 열 번 반복
- * 모델 성능이 좋다 = 예측을 잘한다(정확도 높음) or 훈련 속도가 빠르다



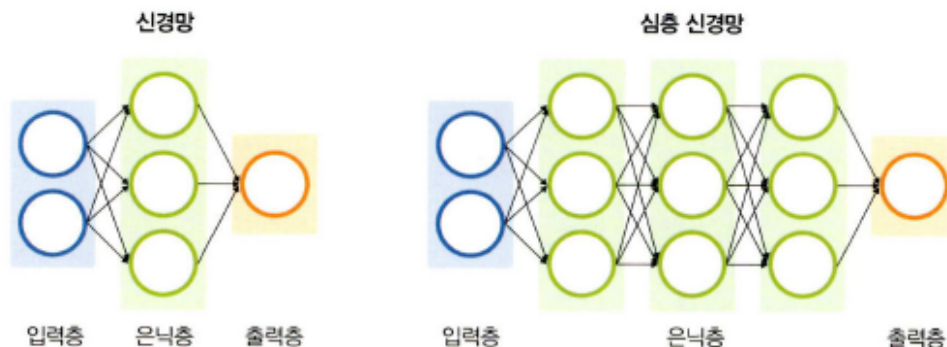
5. 모델(모형) 예측

- 검증 데이터셋을 생성한 모델(모형)에 적용하여 실제로 예측을 진행해보는 단계
- if 예측력 ↓ → 파라미터 튜닝 or 신경망 자체 재설계 필요

• 딥러닝의 핵심 구성 요소

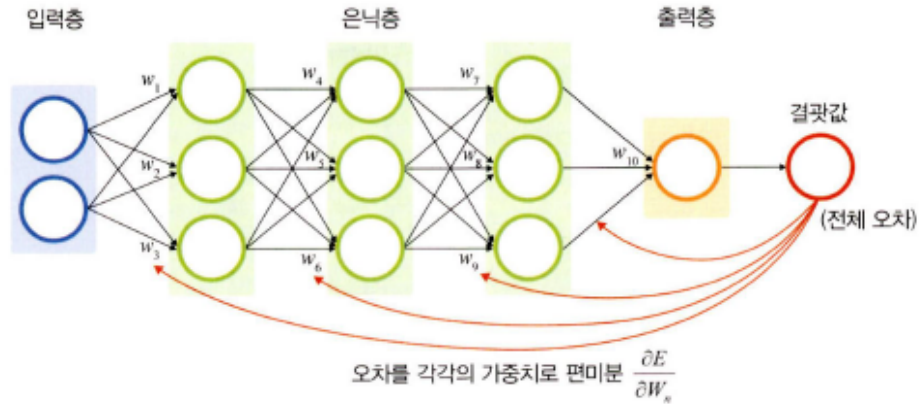
1. 신경망

- 머신 러닝과의 차이점: 심층 신경망을 사용한다는 점
- * 심층 신경망: 은닉층이 두 개 이상인 신경망 - 데이터셋의 어떤 특성들이 중요한지 스스로에게 가르쳐 줄 수 있는 기능 O



1. 역전파

- 가중치 값 업데이트 위함
- 역전파 계산 과정에서 사용되는 미분(오차를 각 가중치로 미분)이 성능에 영향 미치는 주요한 요소
- (파이토치 같은 프레임워크 이용하면 역전파 알고리즘 자동으로 처리해 줌)



1.3.2 딥러닝 학습 알고리즘

• 지도 학습

1. 이미지 분류

- 이미지 또는 비디오상의 객체를 식별하는 컴퓨터 비전 기술
- 주로 합성곱 신경망(CNN) 사용
- 목적에 따라 이미지 분류, 인식, 분할로 분류 가능

- 1) 이미지 분류: 이미지를 알고리즘에 입력하면 그 이미지가 어떤 클래스에 속하는지 알려 주기 때문에 말 그대로 이미지 데이터를 유사한 것끼리 분류할 때 사용함
- 2) 이미지 인식: 사진 분석하여 그 안에 있는 사물의 종류 인식
- 3) 이미지 분할: 영상에서 사물이나 배경 등 객체 간 영역을 픽셀 단위로 구분
ex) X-ray, CT, MRI 등 다양한 의료 영상에서 분할된 이미지 정보 활용하여 질병 진단

2. 시계열 데이터 분류

- 주로 순환 신경망(RNN) 사용
- 시간에 따른 데이터가 있을 때 사용
- 역전파 과정에서 기울기 소멸 문제
→ 개선 - LSTM: 망각 게이트(과거 정보 잊기 위함) + 입력 게이트(현재 정보 기억 위함) + 출력 게이트(최종 결과 위함) 도입

• 비지도 학습

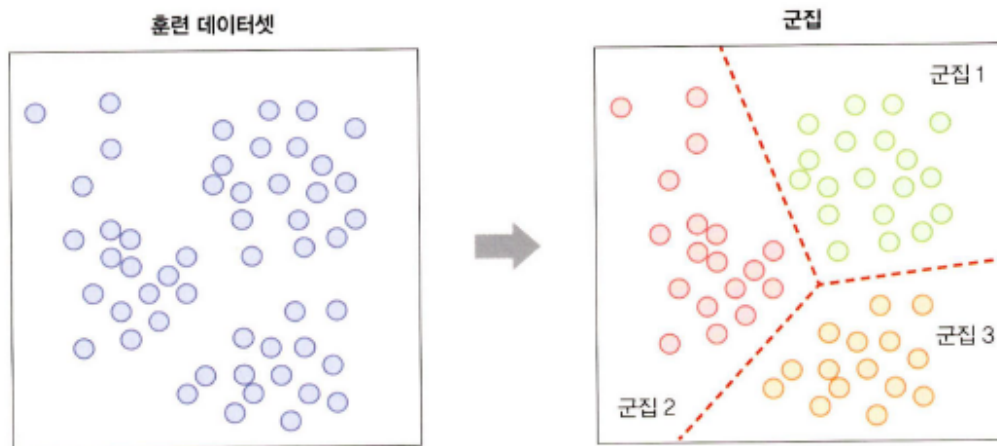
1. 워드 임베딩

- 자연어(사람의 언어)를 컴퓨터가 이해하고 효율적으로 처리하게 하려면 컴퓨터가 이해할 수 있도록 변환하는 것 필요
- 단어를 벡터로 표현
- 주로 워드투벡터, 글로브 사용
ex) 번역, 음성 인식

2. 군집

- 정보 없는 상태에서 데이터 분류

- 목표: 한 클러스터 안의 데이터는 매우 비슷하게 구성하고 다른 클러스터의 데이터와 구분되도록 나누는 것
- * 머신 러닝 군집과 유사하므로 딥러닝과 함께 사용하여 모델 성능 높임



- 전이 학습: 사전에 학습이 완료된 모델(사전 학습 모델)을 가지고 우리가 원하는 학습에 미세 조정 기법 이용해 학습시키는 방법
→ 사전 학습 모델 필요, 이것을 어떻게 활용하는지에 대한 접근 방법 필요
- 0. 사전 학습 모델
 - 풀고자 하는 문제와 비슷, 많은 데이터로 이미 학습이 되어 있는 모델
 - 보통 많은 데이터 구하기 hard, 많은 데이터로 모델 학습시키는 것은 시간, 연산량 ↑
ex) VGG, 인셉션, MobileNet 사용
 - 분석하려는 주제에 맞는 사전 학습 모델 선택하고 활용 필요 (⇒ 5장)
- 강화 학습

구분	유형	알고리즘
지도 학습(supervised learning)	이미지 분류	• CNN • AlexNet • ResNet
	시계열 데이터 분석	• RNN • LSTM
비지도 학습 (unsupervised learning)	군집 (clustering)	• 가우시안 혼합 모델(Gaussian Mixture Model, GMM) • 자기 조직화 지도(Self-Organizing Map, SOM)
	차원 축소	• 오토인코더(AutoEncoder) • 주성분 분석(PCA)
전이 학습(transfer learning)	전이 학습	• 버트(BERT) • MobileNetV2
강화 학습(reinforcement learning)	-	마르코프 결정 과정(MDP)

2장 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

파이토치: 딥러닝 프레임워크. 파이썬 기반. 간결하고 빠른 구현성

- 대상: 넘파이 대체하면서 GPU 이용한 연산 필요한 경우 / 최대한의 유연성과 속도 제공하는 딥러닝 연구 플랫폼 필요한 경우

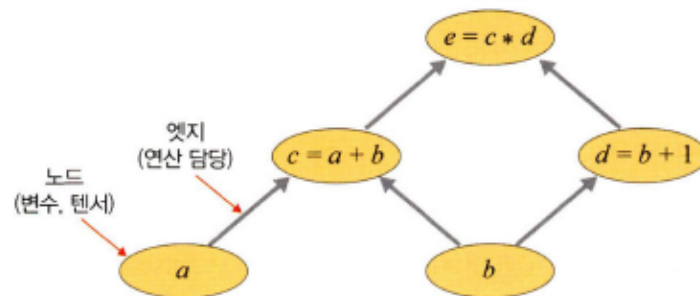
2.1.1 파이토치 특징 및 장점

1. 특징: GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크
 - 1) GPU: 연산 속도 빠르게
 - 내부적으로 CUDA < cuDNN이라는 API 통해 GPU를 연산에 사용 가능
 - 병렬 연산에서 속도는 GPU >> CPU
 - 2) 텐서: 파이토치의 데이터 형태 = 단일 데이터 형식으로 된 자료들의 다차원 행렬
 - 간단한 명령어(변수 뒤에 .cuda())를 추가해서 GPU로 연산 수행하게 할 수 있음
 - 3) 동적 신경망: 훈련 반복할 때마다 네트워크 변경 가능한 신경망
 - ex) 학습 중 은닉층 추가, 제거
 - 연산 그래프 정의하는 것과 동시에 값도 초기화되는 Define by Run 방식 사용 (연산 그래프와 연산 분리하여 생각할 필요 X)
- 벡터, 행렬, 텐서
 - * 인공지능에서 데이터는 벡터로 표현됨
 - 벡터 = 1차원 축(행) = axis 0: [1.0, 1.1, 1.2] 처럼 숫자들의 리스트, 1차원 배열 형태
 - 행렬(matrix) = 2차원 축(열) = axis 1: 행과 열로 표현, 2차원 배열 형태
 - * 가로줄 = 행(row), 세로줄 = 열(column)
 - 텐서 = 3차원 축(채널) = axis 2: 3차원 이상의 배열 형태
- 행렬은 복수 차원 가지는 데이터 레코드의 집합. 이때 하나의 데이터 레코드를 벡터 단독으로 나타낼 때는 하나의 열로 표기됨/ 복수의 데이터 레코드 집합을 행렬로 나타낼 때는 하나의 데이터 레코드가 하나의 행으로 표기됨
 - 텐서는 행렬의 다차원 표현. 같은 크기 행렬이 여러 개 묶여 있는 것. 파이토치에서 텐서 표현은 torch.tensor() 사용

```
import torch
torch.tensor([1., -1.], [1., -1])
# 생성된 텐서의 형태는 다음과 같이 표현됨
tensor([1., -1.,
        [1., -1]])
```


- 연산 그래프

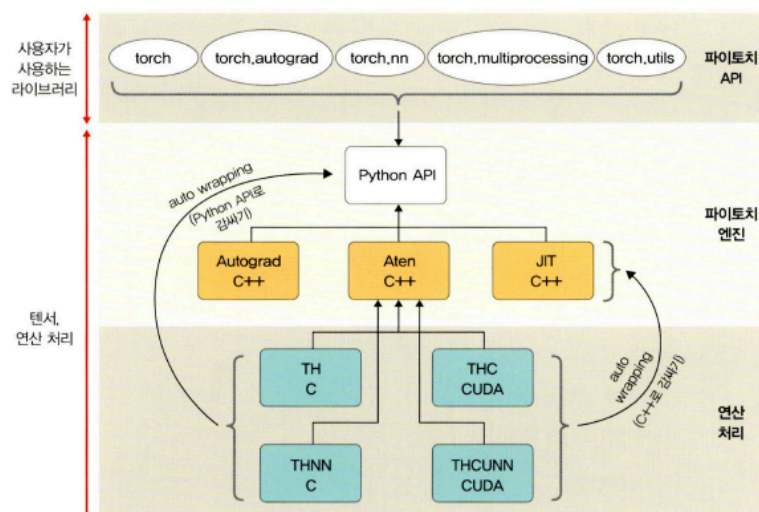
- 연산 그래프는 방향성 O, 변수(ex 텐서) 의미하는 노드 & 연산(ex 곱하기, 더하기)을 담당하는 엣지로 구성. 노드는 변수를 가지고, 각 계산 통해 새 텐서 구성 가능
- 신경망은 연산 그래프 이용하여 계산 수행. 네트워크가 학습될 때 손실 함수의 기울기가 가중치와 바이어스를 기반으로 계산되며, 이후 경사 하강법 사용하여 가중치 업데이트됨. 이때 연산 그래프가 효과적



2. 파이토치 장점

- 단순함(효율적인 계산) - 파이썬 환경과 쉽게 통합, 디버깅 직관적이고 간결
- * 디버깅: 오작동되는 현상을 해결하는 것. 오류들을 찾아내기 위한 테스트 과정
- 성능(낮은 CPU 활용) - 모델 훈련 위한 CPU 사용률 < 텐서플로. 학습 및 추론 속도가 빠르고 다루기 쉬움
- 직관적인 인터페이스 - 텐서플로처럼 잦은 API 변경 없어 배우기 쉬움

2.1.2 파이토치의 아키텍처



크게 위에서부터 API, 파이토치 엔진, 연산 처리 세 계층으로 나누어짐

1. 파이토치 API

- 사용자가 이해하기 쉬운 API 제공해 텐서에 대한 처리와 신경망을 구축하고 훈련할 수

있도록 도움

- 사용자 인터페이스 제공 but 실제 계산 수행 X. 대신 C++ 로 작성된 파이토치 엔진으로 그 작업 전달 역할

- 사용자 편의성 위해 다음 패키지들 제공됨

1) torch: GPU 지원하는 텐서 패키지

- 다차원 텐서 기반으로 다양한 수학적 연산 가능하도록

- CPU뿐만 아니라 GPU에서 연산 가능하므로 빠른 속도, 많은 양 계산 가능

2) torch.autograd: 자동 미분 패키지

- 자동 미분 기술 채택하여 미분 계산 효율적으로 처리함

- 연산 그래프가 즉시 계산(실시간으로 네트워크 수정이 반영된 계산)되기 때문에 사용자는 다양한 신경망 적용해 볼 수 O

3) torch.nn: 신경망 구축 및 훈련 패키지

- 신경망 쉽게 구축하고 사용 가능

- 특히 합성곱 신경망, 순환 신경망, 정규화 등 포함되어 손쉽게 신경망 구축하고 학습시킬 수 O

4) torch.multiprocessing: 파이썬 멀티프로세싱 패키지

- 프로세스 전반 걸쳐 텐서 메모리 공유 가능

- 서로 다른 프로세스에서 동일한 데이터(텐서)에 대한 접근 및 사용 가능

5) torch.utils: DataLoader 및 기타 유틸리티를 제공하는 패키지

- 모델에 데이터를 제공하기 위한 torch.utils.data.DataLoader 모듈을 주로 사용

- 병목 현상 디버깅 위한 torch.utils.bottleneck, 모델 또는 모델의 일부를 검사하기 위한 torch.utils.checkpoint 등의 모듈도 O

2. 파이토치 엔진

Autograd C++, Aten C++, JIT C++, Python API로 구성

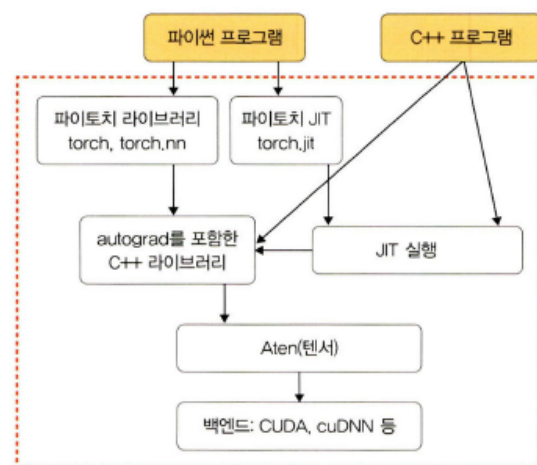
1) Autograd C++: 가중치, 바이어스를 업데이트하는 과정에서 필요한 미분을 자동으로 계산해 주는 역할

2) Aten C++: 텐서 라이브러리 제공

3) JIT C++: 계산 최적화 위한 JIT 컴파일러

* 파이토치 엔진 라이브러리는

C++로 감싼(래핑) 다음 Python API 형태로 제공되기 때문에 사용자들이 손쉽게 모델 구축하고 텐서 사용할 수 있다



3. 연산 처리

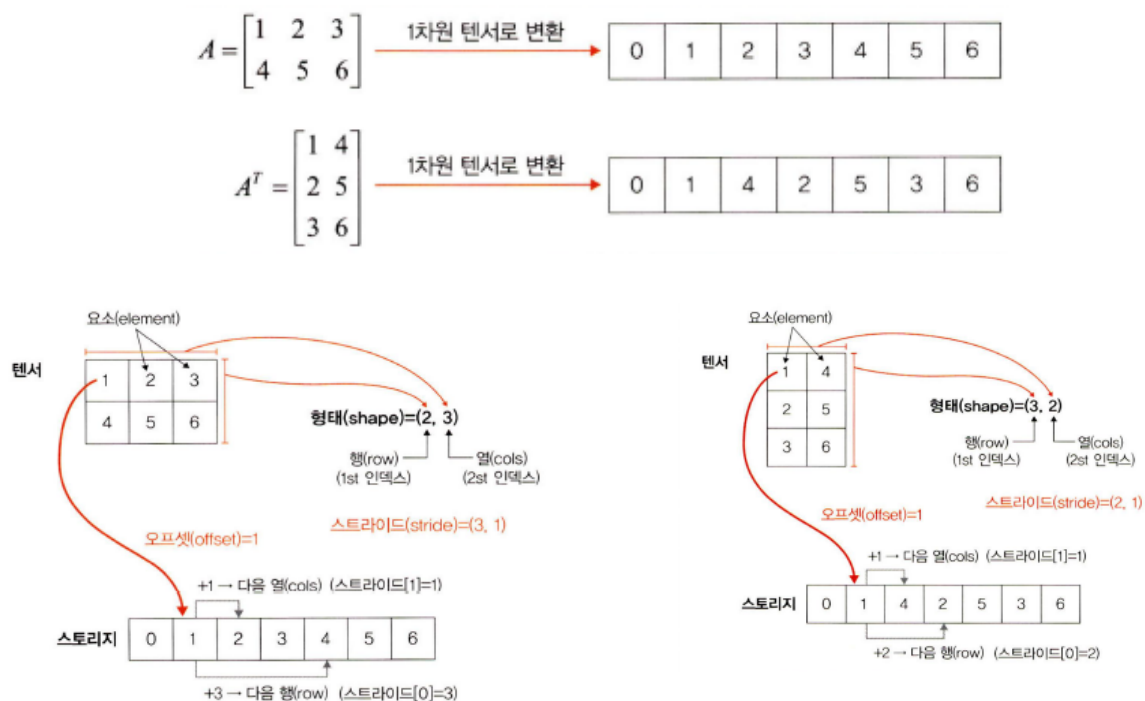
- 가장 아래 계층에 속하는 C 또는 CUDA 패키지는 상위의 API에서 할당된 거의 모든 계산 수행
- 여기서 제공되는 패키지는 CPU와 GPU(TH(토치), THC(토치 CUDA))를 이용하여 효율적인 데이터 구조, 다차원 텐서에 대한 연산 처리함

- 텐서를 메모리에 저장하기

*텐서는 항상 1차원 배열 형태로 메모리에 저장됨. 그 변환된 1차원 배열 = 스토리지
 A 와 A^T 는 다른 형태(shape)지만 스토리지 값들은 같음. 구분 용도로 오프셋과 스트라이드 사용

*오프셋: 텐서에서 첫 번째 요소가 스토리지에 저장된 인덱스

*스트라이드: 각 차원에 따라 다음 요소 얻기 위해 건너뛰기가 필요한 스토리지의 요소 개수. 메모리에서의 텐서 레이아웃 표현하는 것. 요소가 연속적으로 저장되므로 행 중심으로 스트라이드는 항상 1 - 다음 행/열로 이동하기 위해 스토리지에서 몇 칸 움직여야하는지



2.2 파이토치 기초 문법

2.2.1 텐서 다루기

1. 텐서의 인덱스 조작

- 텐서의 자료형

1) torch.FloatTensor: 32비트의 부동 소수점

2) torch.DoubleTensor: 64비트의 부동 소수점

3) torch.LongTensor: 64비트의 부호가 있는 정수

*텐서끼리 다양한 수학 연산 가능. 단, 텐서 간의 타입 다르면 연산 불가능(오류 발생)

2. 텐서의 차원 조작

- 주로 view 이용. 텐서 결합 stack, cat과 차원 교환 t, transpose도 사용

- view - 넘파이의 reshape과 유사/ cat - 다른 길이의 텐서 하나로 병합할 때/

transpose - 행렬의 전치 외에도 차원 순서 변경할 때도 사용

2.2.2 데이터 준비

데이터 호출에는 파이썬 라이브러리(판다스(Pandas))를 이용하는 방법과 파이토치에서 제공하는 데이터를 이용하는 방법이 있다.

1) 데이터가 이미지일 경우(이미지 모델 사용) 분산된 파일에서 데이터를 읽음 → 전처리 → 배치 단위로 분할하여 처리

2) 데이터가 텍스트일 경우(텍스트 모델 사용) 임베딩 과정 → 다른 길이의 시퀀스를 배치 단위로 분할하여 처리

1. 단순하게 파일을 불러와서 사용

- 판다스 라이브러리를 이용하여 JSON, PDF, CSV 등의 파일을 불러오는 방법

- 데이터가 복잡하지 않은 형태라면 단순하고 유용하게 사용될 수 O

2. 커스텀 데이터셋을 만들어서 사용

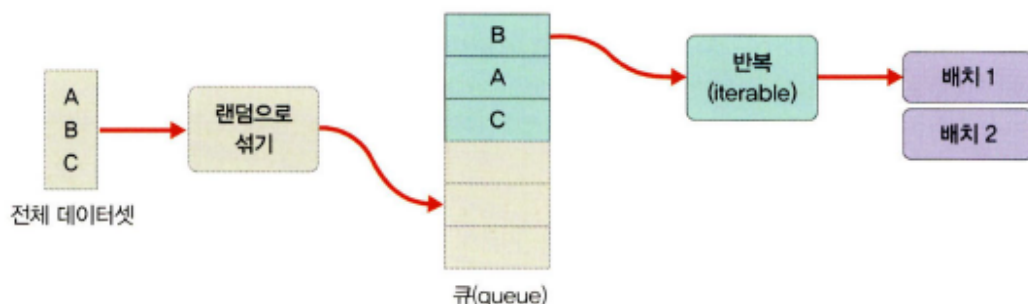
- 데이터를 한 번에 X 나누어 불러서 사용하는 방식

- 대량의 데이터 이용시

• torch.tuils.data.DataLoader

데이터로더 객체는 학습에 사용될 데이터 전체를 보관했다가 모델 학습 할 때 배치 크기 만큼 데이터를 꺼내서 사용

*주의: 데이터 미리 잘라 놓는 것 X. 내부적으로 반복자(iterator)에 포함된 인덱스 이용해 배치 크기만큼 데이터 반환



3. 파이토치에서 제공하는 데이터셋 사용

- 토치비전: 파이토치에서 제공하는 데이터셋들 모여 있는 패키지. MNIST, ImageNet

포함한 유명 데이터셋들 제공

- <https://pytorch.org/vision/0.8/datasets.html>

2.2.3 모델 정의

파이토치에서 모델 정의하기 위해서는 모듈을 상속한 클래스 사용

- 계층: 모듈 또는 모듈 구성하는 한 개의 계층. 합성곱층(convolutional layer), 선형계층(linear layer) 등이 있음

- 모듈: 한 개 이상의 계층이 모여서 구성된 것. 모듈이 모여 새로운 모듈 만들 수 O

- 모델: 최종적으로 원하는 네트워크. 한 개 모듈이 모델 될 수 O

1. 단순 신경망 정의하는 방법

- nn.Module을 상속받지 않는 단순한 모델 생성 시 사용
- 구현 쉽고, 단순

2. nn.Module()을 상속하여 정의하는 방법

- nn.Module을 상속받는 모델은 기본적으로 `__init__()`과 `forward()` 함수 포함함
- `__init__()` - 모델에서 사용될 모듈, 활성화 함수 등 정의
- `forward()` 함수 - 모델에서 실행되어야 하는 연산 정의

3. Sequential 신경망을 정의하는 방법

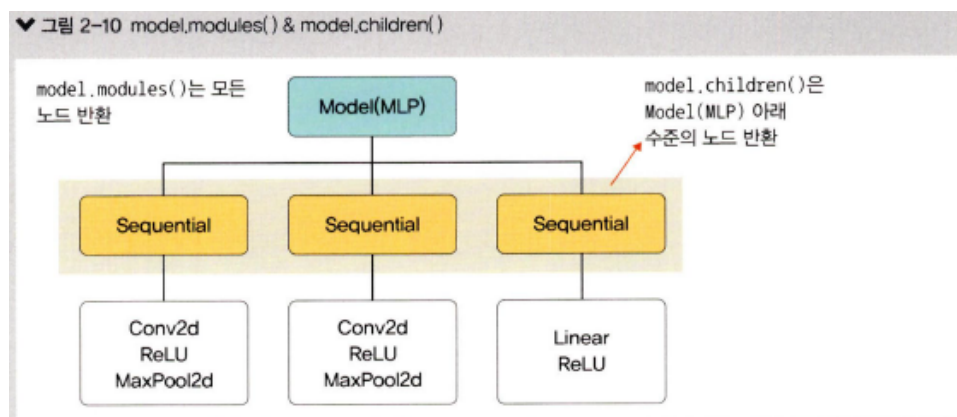
- nn.Sequential 사용시 `__init__()`에서 사용할 네트워크 모델 정의해줌 + `forward()` 함수에서는 모델에서 실행되어야 할 계산을 더 가독성 있게 작성함
- Sequential 객체는 그 안의 각 모듈을 순차적으로 실행해줌
- nn.Sequential은 모델 계층 복잡할수록 효과 ↑

• model.modules()

: 모델의 네트워크에 대한 모든 노드 반환

• model.children()

: 같은 수준의 하위 노드 반환



4. 함수로 신경망 정의하는 방법

- Sequential 이용과 동일

- 함수로 선언 시 변수에 저장해놓은 계층들을 재사용할 수 O
- 모델 복잡해짐 → 복잡한 모델 경우 함수 이용보다 nn.Module() 상속받아 사용하는 것이 편리함
- ReLU, Softmax 및 Sigmoid와 같은 활성화 함수는 모델 정의할 때 지정함

2.2.4 모델의 파라미터 정의

모델을 학습하기 전에 필요한 파라미터들을 정의함. 사전에 정의할 파라미터는 다음과 같음

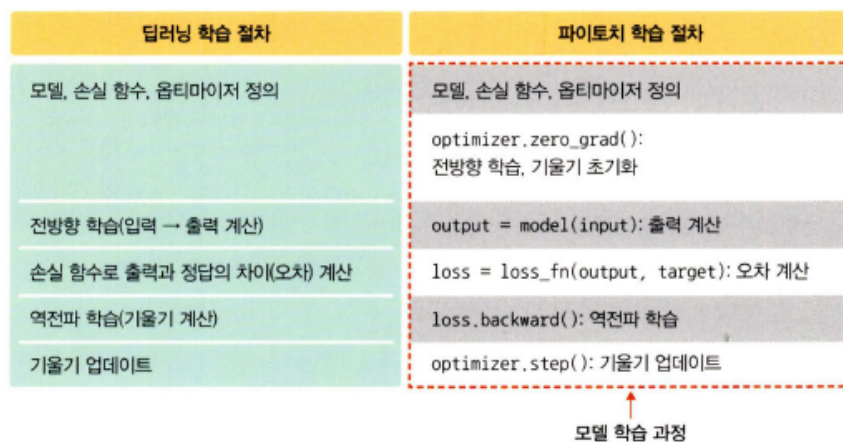
1. 손실함수(loss function): 학습동안 출력과 실제 값(정답) 사이의 오차 측정. $w x + b$ 와 실제 값 y 의 오차 구해 모델의 정확성 측정
 ⇒ 손실함수 예시
 - BCELoss - 이진 분류 위해 사용
 - CrossEntropyLoss - 다중 클래스 분류
 - MSELoss: 회귀 모델에서
2. 옵티마이저(optimizer): 데이터와 손실 함수 바탕으로 모델의 업데이트 방법 결정
 ⇒ 주요 특성
 - step() 매서드 통해 전달받은 파라미터를 업데이트
 - 모델의 파라미터별로 다른 기준(학습률 등)을 적용시킬 수 O
 - torch.optim.Optimizer(params, defaults) - 모든 옵티마이저의 기본이 되는 클래스
 - zero_grad() 매서드 - 옵티마이저에 사용된 파라미터들의 기울기를 0으로 만든다
 - torch.optim.lr_scheduler - 에포크에 따라 학습률 조절
 - 옵티마이저에 사용되는 종류 → 4장
3. 학습률 스케줄러: 미리 지정한 횟수의 에포크를 지날 때마다 학습률을 감소시킴. 학습 초기에는 빠른 학습 진행하다가 전역 최소점 근처에 다다르면 학습률 줄여 최적점 찾아가도록 해줌.
 ⇒ 종류
 - optim.lr_scheduler.LambdaLR - 람다 함수 이용하여 함수 결과를 학습률로 설정
 - optim.lr_scheduler.StepLR - 특정 단계(step)마다 학습률을 감마 비율만큼 감소시킴
 - optim.lr_scheduler.MultiStepLR - StepLR과 비슷하지만 특정 단계 X 지정된 에포크에만 감마 비율로 감소시킴
 - optim.lr_scheduler.ExponentialLR - 에포크마다 이전 학습률에 감마만큼 곱함
 - optim.lr_scheduler.CosineAnnealingLR - 학습률을 코사인 함수의 형태로 변화. 학습률 커지기도 작아지기도 함
 - optim.lr_scheduler.ReduceLROnPlateau - 학습 잘되고 있는지 아닌지에 따라 동적으로 학습률 변화시킬 수 O
4. 지표(metrics): 훈련과 테스트 단계를 모니터링

- 전역 최소점과 최적점
 - 전역 최소점: 오차가 가장 작을 때 = 최종적으로 찾고자하는 것 = 최적점
 - 지역 최소점: 전역 최소점 찾아가는 과정에 만나는 홀(hole)과 같은 것. 옵티마이저가 지역 최소점에서 학습 멈추면 최솟값을 갖는 오차를 찾을 수 없는 문제 발생
 - * 학습 목표: 실제 값과 예측 값 차이를 수치화해 주는 손실 함수의 값을 최소화하는 가중치와 바이어스를 찾는 것

2.2.5 모델 훈련

학습을 시킨다 : $y = wx + b$ 라는 함수에서 w 과 b 의 적절한 값을 찾는다. w 과 b 에 임의의 값 적용하여 시작하며 오차 줄어들어 전역 최소점에 이를 때까지 파라미터(w, b)를 계속 수정

1. `optimizer.zero_grad()` 메서드 이용하여 기울기 초기화
 - 기울기 값에 대해 누적 계산 필요하지 X 경우
 - * 파이토치는 기울기 값 계산 위해 `loss.backward()` 메서드 사용 ← 새로운 기울기 값이 이전 기울기 값에 누적하여 계산됨. RNN 모델 구현할 때는 효과적. 누적 필요 없을 땐 불필요



2. `loss.backward()` 메서드 이용하여 기울기를 자동 계산
 - `loss.backward()`는 배치 반복될 때마다 오차 중첩적으로 쌓임 → 매번 `zero_grad()` 사용하여 미분 값을 0으로 초기화

2.2.6 모델 평가

모델 평가는 함수와 모듈을 이용하는 두 가지 방법 존재

2.2.7 훈련 과정 모니터링

텐서 보드 이용하면 학습에 사용되는 각종 파라미터 값이 어떻게 변화하는지 손쉽게 시각화하여 살펴볼 수 있으며 성능 추적하거나 평가하는 용도로도 사용할 수 있다

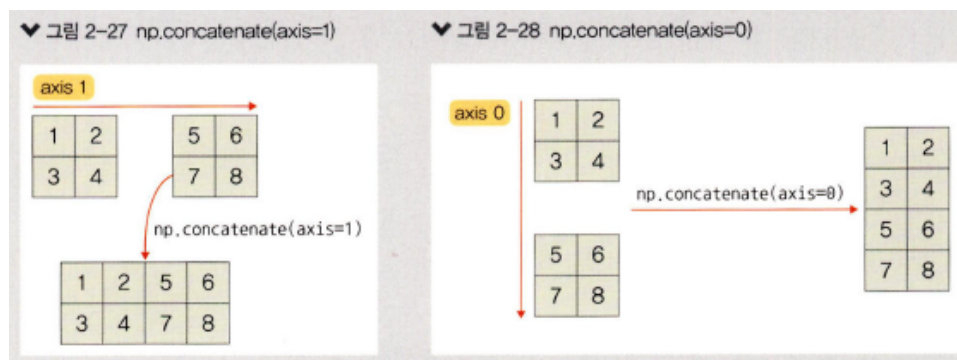
1. 텐서보드를 설정(set up)한다
2. 텐서보드에 기록(write)한다
3. 텐서보드를 사용하여 모델 구조를 살펴본다
 - `model.train()`: 훈련 데이터셋에 사용하며 모델 훈련이 진행될 것을 알림. 이때 dropout 활성화됨
 - `model.eval()`: 모델 평가할 때는 모든 노드를 사용하겠다는 의미로 검증과 테스트 데이터셋에 사용함
 - 이 둘을 선언하면 모델의 정확도 ↑
 - *`model.eval()`에서 `with torch.no_grad()` 사용 이유: 파이토치는 모든 연산과 기울기 값을 저장함. but 검증(혹은 테스트)는 역전파 필요 X 때문에 `with torch.no_grad()` 사용하여 기울기 값 저장하지 않도록

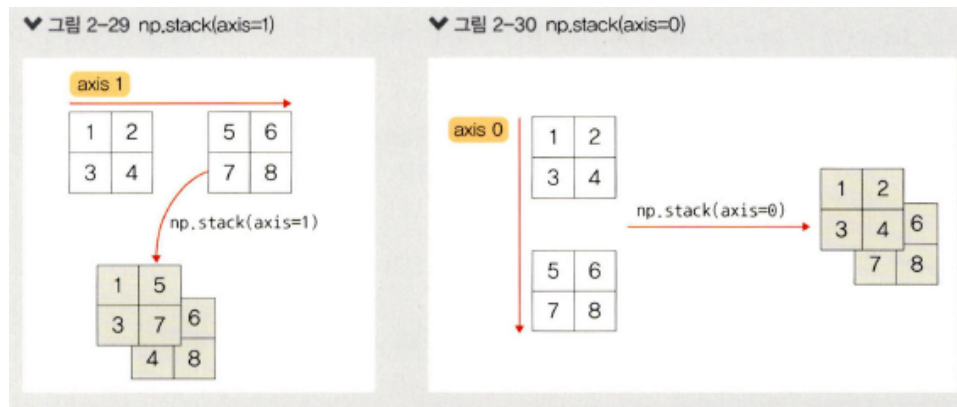
2.4 파이토치 코드 맛보기

- 데이터에 대한 전처리
 - 1) 데이터 파악 : 데이터의 형태 파악 후 숫자로 변환
 - 2) 데이터 변환: `astype()` 메서드 이용하여 범주 특성 갖는 데이터를 범주형 타입으로 변환
 - 파이토치를 이용한 모델 학습 위해 범주형 타입을 센서로 변환

범주형 데이터 → `dataset[category]` → 넘파이 배열(Numpy array) → 텐서(Tensor)

*범주형 데이터(단어) → 숫자(넘파이 배열) 변환 위해 `cat.codes` 사용
- `np.stack` & `np.concatenate`
 - 둘 다 넘파이 객체 합칠 때 사용하는 메서드
 - 차원의 유지 여부에 대한 차이
 - 1) `np.concatenate` - 선택한 축(axis) 기준으로 두 개의 배열 연결
 - 2) `np.stack` - 배열들을 새로운 축으로 합침(두 배열의 차원 동일해야 함)





- `ravel()`, `reshape()`, `flatten()`
 - 텐서의 차원 바꿀 때
 - 2차원 텐서 → 1차원
- 워드 임베딩
 - 높은 차원의 임베딩일수록 단어 간의 세부적인 관계를 잘 파악할 수 있다
 - 단일 숫자로 변환된 넘파이 배열을 N차원으로 변경하여 사용
 - 보통 임베딩 크기 - 칼럼의 고유 값 수 / 2
- 모델의 네트워크 생성
 - 1) 클래스 형태로 구현되는 모델은 `nn.Module`을 상속받음
 - 2) `__init__()` - 모델에서 사용될 파라미터와 신경망 초기화 용도(객체 생성시 자동으로 호출)
 - `self`: 첫 번째 파라미터는 `self` 지정해야함. 자기자신
 - `embedding_size`: 범주형 칼럼의 임베딩 크기
 - `output_size`: 출력층의 크기
 - `layers`: 모든 계층에 대한 목록
 - `p`: 드롭아웃(기본값 0.5)
 - 3) `super().__init__()` - 부모 클래스 접근할 때. `super`는 `self` 사용 x
 - 4) 모델의 네트워크 계층 구축 위해 for문. 각 계층을 `all_layers` 목록에 추가
 - Linear: 선형 계층. 입력 데이터에 선형 변환 진행. $y = Wx + b$
 - * y : 선형 계층의 출력 값, W : 가중치, x : 입력 값, b : 바이어스
 - ReLU: 활성화 함수로 사용
 - BatchNorm1d: 배치정규화 용도
 - Dropout: 과적합 방지
 - 5) `forward()` 함수 - 학습 데이터를 입력 받아서 연산 진행. 모델 객체를 데이터와 함께 호출하면 자동으로 실행됨
- 파이토치는 GPU에 최적화된 딥러닝 프레임워크. But GPU가 없다면 CPU 사용할 수 있도록 지정해 주어야함

- 성능 평가 지표 - 정확도(accuracy), 재현율(recall), 정밀도(precision), F1-스코어(F1-score)
 - True Positive: 모델(분류기)이 '1'이라고 예측했는데 실제 값도 '1'인 경우
 - True Negative: 모델(분류기)이 '0'이라고 예측했는데 실제 값도 '0'인 경우
 - False Positive: 모델(분류기)이 '1'이라고 예측했는데 실제 값은 '0'인 경우. Type I 오류
 - False Negative: 모델(분류기)이 '0'이라고 예측했는데 실제 값은 '1'인 경우. Type II 오류
- 1) 정확도 : 전체 예측 건수에서 정답을 맞힌 건수의 비율. 정답이 긍정이든 부정이든 상관 X

$$(TruePositive + TrueNegative) / (TruePositive + TrueNegative + FalsePositive + FalseNegative)$$
- 2) 재현율: 실제로 정답이 1이라고 할 때 모델도 1로 예측한 비율. 따라서 처음부터 데이터가 1일 확률이 적을 때 사용하면 좋다

$$TruePositive / (TruePositive + FalseNegative)$$
- 3) 정밀도: 모델이 1이라고 예측한 것 중에서 실제로 정답이 1인 비율

$$TruePositive / (TruePositive + FalsePositive)$$
- 4) F-1스코어: 일반적으로 정밀도와 재현율은 트레이드오프 관계. 이 문제를 해결하기 위해 정밀도와 재현율의 조화 평균 이용한 것

$$2 * Precision * Recall / (Precision + Recall)$$